

untangle::actuator

Generated by Doxygen 1.18.0

1 actuator	1
2 Topic Index	3
2.1 Topics	3
3 Namespace Index	5
3.1 Namespace List	5
4 Class Index	7
4.1 Class List	7
5 File Index	9
5.1 File List	9
6 Topic Documentation	11
6.1 namespace untangle: functions	11
6.1.1 Detailed Description	11
6.1.2 Function Documentation	11
6.1.2.1 connect()	11
6.1.2.2 bind() [1/2]	12
6.1.2.3 bind() [2/2]	13
7 Namespace Documentation	15
7.1 untangle Namespace Reference	15
7.1.1 Detailed Description	15
7.1.2 Function Documentation	15
7.1.2.1 last_arg()	15
8 Class Documentation	17
8.1 untangle::actuator< action_t > Struct Template Reference	17
8.1.1 Detailed Description	18
8.1.2 Member Typedef Documentation	19
8.1.2.1 actions_t	19
8.1.2.2 results_t	19
8.1.3 Member Function Documentation	20
8.1.3.1 operator=()	20
8.1.3.2 type()	20
8.1.3.3 reset()	20
8.1.3.4 invoke_callback()	21
8.1.3.5 operator()()	22
8.1.3.6 invoke_action()	22
8.1.3.7 add() [1/2]	22
8.1.3.8 add() [2/2]	23
8.1.3.9 remove() [1/2]	23
8.1.3.10 remove() [2/2]	24
8.1.3.11 is_connected()	24

8.1.3.12 <code>has_action()</code>	24
8.2 <code>untangle::invalid_action</code> Struct Reference	25
8.2.1 Detailed Description	25
8.2.2 Constructor & Destructor Documentation	25
8.2.2.1 <code>invalid_action()</code>	25
8.2.3 Member Function Documentation	26
8.2.3.1 <code>what()</code>	26
9 File Documentation	27
9.1 <code>actuator.hpp</code>	27
Index	31

Chapter 1

actuator

An actuator is a generic callable that may invoke a list of actions(callables of type `std::function<...>`).

Its goal is to provide a light and simple mechanism to connect components that want to communicate with each other. It works similar as the "signal and slots" mechanisms from other frameworks, but much simplified.

Modern programming languages do not offer class intrinsic interfaces that would improve significantly how objects are connected and how they communicate. Instead, languages, like C++, are using external interfaces, that are not only cluttering the code, but they don't feel "natural" as they are opposed to what an interface is in reality. Basically the objects and interfaces are not separated, and the interface is intrinsic to the object(think on the cogs in a mechanism). A generic callable is an attempt to mitigate this problem. `async` implementation is such use case.

An actuator provides the results of all actions in a vector, in the same order as the actions were added.

Example: how to use actuator instead of polymorphism

```
void rotate_shapes(const std::vector<shape*>& shapes, int angle)
{
    for (const auto& s : shapes)
    {
        s->rotate(angle);
    }
}

std::shared_ptr<triangle> t(new triangle);
std::shared_ptr<circle> c(new circle);
std::shared_ptr<square> s(new square);

// using polymorphism
std::vector<shape*> shapes;
shapes.push_back(t.get());
shapes.push_back(c.get());
shapes.push_back(s.get());

rotate_shapes(shapes, 10);

// using actuator
auto action1 = untangle::bind(t, &triangle::rotate);
auto action2 = untangle::bind(c, &circle::rotate);
auto action3 = untangle::bind(s, &square::rotate);
auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate(20);
```

For convenience there are provided helpers methods to "connect" to an initial list of "actions", or to create bindings to class methods.

Please check the manual in [doc/refman.pdf](#) for further references.

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions [11](#)

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	Interface to untangle::actuator functor	15
--------------------------	---	--------------------

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::actuator< action_t >	
An actuator is a functor that can trigger a dynamic list of actions (of type std::↵ function<...>)	17
untangle::invalid_action	
Invalid action exception	25

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

actuator.hpp	27
--	----

Chapter 6

Topic Documentation

6.1 namespace untangle: functions

Functions

- `template<typename action_t, typename... Actions>`
`actuator< action_t > untangle::connect (action_t &A1, Actions &... An)`
Creates an actuator holding an initial list of actions.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↔`
`T>::type>>`
`action_t untangle::bind (const std::shared_ptr< class_t > &obj, T class_t::*method)`
Binding to a class function member.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↔`
`T>::type>>`
`action_t untangle::bind (class_t *obj, T class_t::*method)`
Binding to a class method.

6.1.1 Detailed Description

6.1.2 Function Documentation

6.1.2.1 connect()

```
template<typename action_t, typename... Actions>
actuator< action_t > untangle::connect (
    action_t & A1,
    Actions &... An)
```

Creates an actuator holding an initial list of actions.

Parameters

A1	- The first action. It is specified as <code>std::function<...></code> .
----	--

An	- Any number of further actions, of the same type as A1.
----	--

Returns

An [actuator](#).

Example:

```
void rotate_shapes(const std::vector<shape*>& shapes, int angle) {
    for (const auto& s : shapes) {
        s->rotate(angle);
    }
}

auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate(20);
```

6.1.2.2 bind() [1/2]

```
template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<T>::<
type>>>
action_t untangle::bind (
    const std::shared_ptr< class_t > & obj,
    T class_t::* method)
```

Binding to a class function member.

It returns a `std::function(lambda)` that wraps the function member. It may be used to provide an action for [connect\(\)](#) or [actuator::add\(\)](#).

Remarks

It requires a shared pointer to the class type. A `std::weak_ptr` to it is captured internally in a lambda, so the binding can always check whether the shared object is still alive without keeping it alive itself. Therefore, it is safe to use actions provided by this binding inside an [actuator](#), and safe for the action to outlive the caller's shared pointer.

Parameters

obj	- Class object.
method	- Pointer to function member. It is specified as <code>&<class type>::<function member></code>

Returns

`action_t` - A `std::function` that wraps the pointer to function member.

Remarks

If the class object gets invalid, invoking this binding will throw an exception of type [invalid_action](#).

6.1.2.3 bind() [2/2]

```
template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<T>::↵  
type>>  
action_t untangle::bind (  
    class_t * obj,  
    T class_t::* method)
```

Binding to a class method.

Attention

It is not safe to use this binding when the pointed-to object may be destroyed. A null pointer is detected and reported as a dead action, but a pointer to an already destroyed object can not be distinguished from a valid one. Prefer the overload taking a `std::shared_ptr`.

Remarks

It is provided for convenience of use: within a class it is safe to create bindings through this pointer.

Binding a null pointer produces an action that is always dead: invoking it throws [invalid_action](#), and an [actuator](#) drops it.

Parameters

obj	- Pointer to class.
method	- Pointer to function member. It is specified as <code>&<class type>::<function member></code>

Returns

action_t - A `std::function` that wraps the pointer to function member.

Chapter 7

Namespace Documentation

7.1 untangle Namespace Reference

Interface to `untangle::actuator` functor.

Classes

- struct `invalid_action`
Invalid action exception.
- struct `actuator`
An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

Functions

- `template<typename... Args>`
`auto last_arg (Args &&... args)`
Get last argument of params pack;.
- `template<typename action_t, typename... Actions>`
`actuator< action_t > connect (action_t &A1, Actions &... An)`
Creates an actuator holding an initial list of actions.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵`
`T>::type>>`
`action_t bind (const std::shared_ptr< class_t > &obj, T class_t::*method)`
Binding to a class function member.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵`
`T>::type>>`
`action_t bind (class_t *obj, T class_t::*method)`
Binding to a class method.

7.1.1 Detailed Description

Interface to `untangle::actuator` functor.

7.1.2 Function Documentation

Generated by Doxygen

7.1.2.1 last_arg()

Template Parameters

Args	params template types
------	-----------------------

Parameters

args	params
------	--------

Returns

last param value in case it exist, otherwise 0.

Chapter 8

Class Documentation

8.1 untangle::actuator< action_t > Struct Template Reference

An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

```
#include <actuator.hpp>
```

Public Types

- using `actions_t` = `std::list<action_t*>`
Actions container type.
- using `results_t` = `std::vector<typename result_t::type>`
Results container type.

Public Member Functions

- `actuator & operator=` (`const actuator &other`)=default
Assignment operator.
- `action_t type` () const
Helper method to access the action type of this object.
- void `reset` ()
Remove all actions and any stored results.
- `template<typename last_t>`
void `invoke_callback` (`last_t &last`)
Invoke the trailing callback argument, if the invocation has one.
- `template<typename... Args>`
void `operator()` (`Args &&... args`)
The call operator.
- `template<typename... Args>`
void `invoke_action` (`const std::string &name, Args &&... args`)
Invokes one single action associated with a key.
- void `add` (`action_t *action`)
Add an action to the actions list.
- void `add` (`const std::string &name, action_t *action`)
Add action to the actions map associated with a name.

- void `remove` (const action_t *action)
Remove an action from the actions list.
- void `remove` (const std::string &name)
Remove an action from actions map.
- bool `is_connected` () const
Check if this actuator is "connected" with other actions.
- bool `has_action` (const std::string &name) const
Check if there is certain named action.

Public Attributes

- `actions_t` actions
Actions list.
- `actions_map_t` actions_map
Named actions map.
- `results_t` results
Actions return values list.

8.1.1 Detailed Description

```
template<typename action_t>
struct untangle::actuator< action_t >
```

An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

Remarks

An actuator object can be constructed with an initial list of actions by `connect()`.

Callback convention: when the last argument of an invocation can be called with the action return type `R` and returns nothing, it is not passed on as a plain argument alone – it is also treated as a completion callback. The action is invoked with the full argument list as usual, and the callback is then invoked with the action return value. Any void returning callable qualifies: a `std::function<void(R)>`, a lambda taking `R`, a function pointer. It applies to `operator()` and to `invoke_action()` alike. Actions returning void have no result to report, so no callback is invoked for them.

Warning

The return type is what separates a callback from an argument the action means to consume itself. A trailing callable that returns a value – a transform, a comparator – is left alone, and by the same rule a callback written to return something is silently not invoked. A callback returns nothing.

Remarks

Argument convention: an action must not take ownership of the arguments it is invoked with. `operator()` forwards one argument pack to every action in the list, and an rvalue can be moved from only once, so an action that consumes an argument leaves the actions after it holding a moved-from object – an empty `std::function` for a callback a later action means to call itself. The arguments are forwarded rather than copied so that a single action pays nothing for the broadcast; what that buys has to be respected by the actions.

Warning

An action does not have to move an argument in its body to consume it: a parameter taken by value is move constructed from an rvalue by the call itself. Take arguments by reference or const reference, or invoke with lvalues, whenever more than one action is connected. The behaviour is otherwise unpredictable, and it is the caller and the action signatures together that decide it – an actuator cannot detect the violation.

Remarks

[invoke_action\(\)](#) invokes one single action, so nothing follows it and the convention does not constrain it.

Template Parameters

<code>action_↔ _t</code>	Action type. It is specified as <code>std::function<...></code> .
------------------------------	---

8.1.2 Member Typedef Documentation

8.1.2.1 actions_t

```
template<typename action_t>
using untangle::actuator< action_t >::actions_t = std::list<action_t*>
```

Actions container type.

Remarks

The elements stored are of pointer type, that is required to implement the [remove\(\)](#) operation. `std::function` supports only equality operator for nullptr (two `std::function(s)` can not compare).

8.1.2.2 results_t

```
template<typename action_t>
using untangle::actuator< action_t >::results_t = std::vector<typename result_t::type>
```

Results container type.

It holds the return values of the actions that have a non-void return type. Upon the actuator invocation, the returns can be extracted from [results](#).

8.1.3 Member Function Documentation

8.1.3.1 operator=()

```
template<typename action_t>
actuator & untangle::actuator< action_t >::operator= (
    const actuator< action_t > & other) [default]
```

Assignment operator.

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
untangle::actuator<decltype(actuator_rotate.type())> actuator_rotate_1 = actuator_rotate;
actuator_rotate_1(20);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

8.1.3.2 type()

```
template<typename action_t>
action_t untangle::actuator< action_t >::type () const [inline]
```

Helper method to access the action type of this object.

Note

Intended to be used in expressions by `decltype(<actuator instance>.type())`

Returns

`action_t`

8.1.3.3 reset()

```
template<typename action_t>
void untangle::actuator< action_t >::reset () [inline]
```

Remove all actions and any stored results.

After this call the actuator is empty: `actuator::is_connected()` returns false.

8.1.3.4 invoke_callback()

```
template<typename action_t>
template<typename last_t>
void untangle::actuator< action_t >::invoke_callback (
    last_t & last) [inline]
```

Invoke the trailing callback argument, if the invocation has one.

Does nothing unless last_t can be called with the action return type and returns nothing, so any other trailing argument is left alone. The test is on what the type can do, not on what it is, so a plain lambda qualifies as well as a std::function. See the callback convention on [actuator](#).

Remarks

The single condition does the work of two tests: (1) whether last can be called with the action return type at all, and (2) whether that call returns nothing. Spelled out as traits they need two nested if constexpr:

```
// (1)
if constexpr (std::is_invocable_v<last_t&, typename result_t::type>) {
    // (2)
    if constexpr (std::is_void_v<std::invoke_result_t<last_t&, typename result_t::type>)>) {
        last(results.back());
    }
}
```

Both are folded into the one expression instead, each in a different part of it:

```
requires { requires std::is_void_v<decltype( last(results.back()) )>; }
//      \_____(2)_____/      \_____(1)_____/
```

Remarks

(1) Invocability comes for free. To form decltype(last(results.back())) the compiler must first form the call last(...). If last is an int, that expression does not exist – and inside a requires-expression a substitution failure yields false instead of being ill formed, the same property concepts are built on. So invocability is never tested separately: it is a precondition of the return type question, and failing it is what makes the whole condition false.

(2) The return type test is the inner requires, and the doubled keyword is not a typo. requires { expr; } asks only whether expr is well formed, and std::is_void_v<...> is well formed when it is false too – that spelling would let a value returning callable through. The second keyword makes it a nested requirement: a condition that has to hold, not merely compile.

The trait spelling has to nest, which is part of why it is not the one used here: the two if constexpr cannot be joined with &&. && short circuits evaluation, not instantiation, so std::invoke_result_t is formed whatever the first trait says, and for a type that is not invocable at all it has no ::type – a hard error rather than a false result.

The caller must copy the callback out of the argument pack before invoking the action: an action taking it by value moves from the caller's std::function, which would leave this an empty function to call.

Template Parameters

last↔ _t	Type of the last argument, as returned by last_arg() .
-------------	--

Parameters

last	- Last argument value, copied before the action was invoked.
------	--

8.1.3.5 operator>()()

```
template<typename action_t>
template<typename... Args>
void untangle::actuator< action_t >::operator() (
    Args &&... args) [inline]
```

The call operator.

Actions in the [actuator::actions](#) list are triggered by invoking the call operator.

Parameters

args	- Arguments list must match the action arity. A trailing callback is invoked once per action, with that action's return value; see the convention on actuator .
------	---

Warning

The same argument pack is forwarded to every action in the list, so the argument convention on [actuator](#) applies here in full: an action that consumes an argument leaves the actions after it with a moved-from object.

8.1.3.6 invoke_action()

```
template<typename action_t>
template<typename... Args>
void untangle::actuator< action_t >::invoke_action (
    const std::string & name,
    Args &&... args) [inline]
```

Invokes one single action associated with a key.

Parameters

name	- Key associated with the action.
args	- Arguments list must match the action arity. A trailing callback is invoked with the action's return value; see the convention on actuator .

8.1.3.7 add() [1/2]

```
template<typename action_t>
void untangle::actuator< action_t >::add (
    action_t * action) [inline]
```

Parameters

action	- Action to be added.
--------	-----------------------

Example:

```

const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).Times(2).WillRepeatedly(testing::Return());
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate.add(&action1);
actuator_rotate(20);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());

```

8.1.3.8 add() [2/2]

```

template<typename action_t>
void untangle::actuator< action_t >::add (
    const std::string & name,
    action_t * action) [inline]

```

Add action to the actions map associated with a name.

Parameters

name	- Name of the action.
action	- Action to be added.

8.1.3.9 remove() [1/2]

```

template<typename action_t>
void untangle::actuator< action_t >::remove (
    const action_t * action) [inline]

```

Remove an action from the actions list.

An invalid action (empty std::function) is implicitly removed when `operator()()` is invoked.

Parameters

action	- Action to be removed.
--------	-------------------------

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).Times(0);
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate.remove(&action1);
actuator_rotate(50);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

8.1.3.10 remove() [2/2]

```
template<typename action_t>
void untangle::actuator< action_t >::remove (
    const std::string & name) [inline]
```

Remove an action from actions map.

Parameters

name	- Name of the action to remove.
------	---------------------------------

8.1.3.11 is_connected()

```
template<typename action_t>
bool untangle::actuator< action_t >::is_connected () const [inline]
```

Check if this actuator is "connected" with other actions.

Returns

- true - if the `actuator::actions` list is not empty.
- false - if the `actuator::actions` list is empty.

8.1.3.12 has_action()

```
template<typename action_t>
bool untangle::actuator< action_t >::has_action (
    const std::string & name) const [inline]
```

Check if there is certain named action.

Parameters

name	- Name associated with the action.
------	------------------------------------

Returns

- true - if name can be found in [actuator::actions_map](#)
- false - if name can not be found in [actuator::actions_map](#)

The documentation for this struct was generated from the following file:

- [actuator.hpp](#)

8.2 untangle::invalid_action Struct Reference

Invalid action exception.

```
#include <actuator.hpp>
```

Public Member Functions

- [invalid_action](#) (std::string text)
Construct a new invalid action object.
- const char * [what](#) () const noexcept override
The message text describing the reason of this exception.

8.2.1 Detailed Description

Invalid action exception.

Remarks

An action may be provided as a binding to a class function member, by using [untangle::bind\(\)](#). When the class object gets invalid, invoking the action will raise an exception to this type.

8.2.2 Constructor & Destructor Documentation

8.2.2.1 invalid_action()

```
untangle::invalid_action::invalid_action (  
    std::string text) [inline], [explicit]
```

Construct a new invalid action object.

Parameters

text	- A message text, describing the reason of this exception.
------	--

8.2.3 Member Function Documentation

8.2.3.1 what()

```
const char * untangle::invalid_action::what () const    [inline], [override], [noexcept]
```

The message text describing the reason of this exception.

Returns

const char* - The message text.

The documentation for this struct was generated from the following file:

- actuator.hpp

Chapter 9

File Documentation

9.1 actuator.hpp

```
00001 // Copyright (c) 2018 Nicolae Popescu. MIT License.
00002
00006 #pragma once
00007
00008 #include <exception>
00009 #include <functional>
00010 #include <iostream>
00011 #include <list>
00012 #include <map>
00013 #include <memory>
00014 #include <string>
00015 #include <type_traits>
00016 #include <utility>
00017 #include <vector>
00018
00019 namespace untangle {
00020 // exception
00029 struct invalid_action : std::exception {
00035     explicit invalid_action(std::string text) : message(std::move(text)) {}
00036
00042     const char* what() const noexcept override { return message.c_str(); }
00043
00044 private:
00045     std::string message;
00046 };
00047
00055 template <typename... Args>
00056 auto last_arg(Args&&... args) {
00057     if constexpr (sizeof...(Args) <= 0) {
00058         return 0;
00059     } else {
00060         return std::get<sizeof...(Args) - 1>(std::forward_as_tuple(args...));
00061     }
00062 }
00063
00102 template <typename action_t>
00103 struct actuator final {
00111     using actions_t = std::list<action_t*>;
00112     using actions_map_t = std::map<std::string, action_t*>;
00113     using result_t = std::conditional<std::is_void<typename action_t::result_type>::value, int,
00114                                     typename action_t::result_type>;
00121     using results_t = std::vector<typename result_t::type>;
00122
00123     actions_t actions;
00124     actions_map_t actions_map;
00125     results_t results;
00126
00127     actuator() = default;
00128     actuator(const actuator&) = default;
00129     actuator(actuator&&) noexcept = default;
00130     ~actuator() = default;
00137     actuator& operator=(const actuator& other) = default;
00138     actuator& operator=(actuator&&) noexcept = default;
00139
00147     action_t type() const { return nullptr; }
00148
00154     void reset() {
```

```

00155     actions.clear();
00156     actions_map.clear();
00157     results.clear();
00158 }
00159
00215 template <typename last_t>
00216 void invoke_callback(last_t& last) {
00217     if constexpr (requires { requires std::is_void_v<decltype(last(results.back()))>; }) {
00218         last(results.back());
00219     }
00220 }
00221
00234 template <typename... Args>
00235 void operator()(Args&&... args) {
00236     results.clear();
00237
00238     // Copied up front, before any action can move the callback out of the argument pack.
00239     auto last = last_arg(args...);
00240
00241     // Dead bindings are collected here and dropped after the loop.
00242     // The actuator does not own the actions it points at, so it must never
00243     // write through action_t* into a std::function belonging to the caller.
00244     std::vector<action_t*> dead_actions;
00245
00246     for (const auto& action : actions) {
00247         // A null pointer or an empty std::function can never be invoked. Calling an
00248         // empty one throws std::bad_function_call, which is not an invalid_action and
00249         // would escape this operator, so drop it instead of invoking it.
00250         if (action == nullptr || !*action) {
00251             dead_actions.push_back(action);
00252             continue;
00253         }
00254
00255         try {
00256             if constexpr (std::is_same_v<typename action_t::result_type, void>) {
00257                 (*action)(std::forward<Args>(args)...);
00258             } else {
00259                 results.push_back((*action)(std::forward<Args>(args)...));
00260                 invoke_callback(last);
00261             }
00262         } catch (const invalid_action& ia) {
00263             std::cout << ia.what() << std::endl;
00264             dead_actions.push_back(action);
00265         }
00266     }
00267
00268     for (const auto& dead_action : dead_actions) {
00269         actions.remove(dead_action);
00270     }
00271 }
00272
00280 template <typename... Args>
00281 void invoke_action(const std::string& name, Args&&... args) {
00282     results.clear();
00283
00284     // Copied up front, before the action can move the callback out of the argument pack.
00285     auto last = last_arg(args...);
00286     const auto& it = actions_map.find(name);
00287     if (it != actions_map.end()) {
00288         try {
00289             if constexpr (std::is_same_v<typename action_t::result_type, void>) {
00290                 (*it->second)(std::forward<Args>(args)...);
00291             } else {
00292                 results.push_back((*it->second)(std::forward<Args>(args)...));
00293                 invoke_callback(last);
00294             }
00295         } catch (const invalid_action& ia) {
00296             std::cout << ia.what() << std::endl;
00297             actions_map.erase(name);
00298         }
00299     }
00300 }
00301
00310 void add(action_t* action) { actions.push_back(action); }
00311
00318 void add(const std::string& name, action_t* action) { actions_map.emplace(name, action); }
00319
00330 void remove(const action_t* action) {
00331     actions.remove_if([&action](const auto& a) { return (action == a); });
00332 }
00333
00339 void remove(const std::string& name) { actions_map.erase(name); }
00340
00347 bool is_connected() const { return !actions.empty() || !actions_map.empty(); }
00348
00356 bool has_action(const std::string& name) const {
00357     return actions_map.find(name) != actions_map.end();

```

```

00358 }
00359 };
00360
00375 template <typename action_t, typename... Actions>
00376 actuator<action_t> connect(action_t& A1, Actions&... An) {
00377     using actuator_t = untangle::actuator<action_t>;
00378     actuator_t actuator;
00379     actuator.actions = {&A1, &An...};
00380
00381     // remove empty actions
00382     actuator.actions.remove_if([](const auto& action) { return (*action == nullptr); });
00383     return actuator;
00384 }
00385
00386 template <typename actuator_t>
00387 void remove_empty_actions(actuator_t& actuator) {
00388     std::vector<typename actuator_t::actions_map_t::const_iterator> removable_iterators;
00389     for (auto it = actuator.actions_map.begin(); it != actuator.actions_map.end(); ++it) {
00390         if (it->second == nullptr) {
00391             removable_iterators.push_back(it);
00392         }
00393     }
00394     for (auto it : removable_iterators) {
00395         actuator.actions_map.erase(it);
00396     }
00397 }
00398
00399 template <typename key_t, typename action_t, typename... Actions>
00400 actuator<action_t> connect(std::pair<key_t, action_t*> A1, Actions... An) {
00401     using actuator_t = untangle::actuator<action_t>;
00402     actuator_t actuator;
00403     actuator.actions_map = {A1, An...};
00404
00405     // remove empty actions
00406     remove_empty_actions(actuator);
00407     return actuator;
00408 }
00409
00410 // generic helpers to remove const qualifier from a function type,
00411 // for instance const member functions
00412 template <typename T>
00413 struct function_remove_const;
00414
00415 template <typename R, typename... Args>
00416 struct function_remove_const<R(Args...)> {
00417     using type = R(Args...);
00418 };
00419
00420 template <typename R, typename... Args>
00421 struct function_remove_const<R(Args...) const> {
00422     using type = R(Args...);
00423 };
00424
00428
00450 template <typename class_t, typename T,
00451         typename action_t = std::function<typename function_remove_const<T>::type>
00452 action_t bind(const std::shared_ptr<class_t>& obj, T class_t::* method) {
00453     return [wp = std::weak_ptr<class_t>(obj), method](auto&&... args) ->
00454         typename action_t::result_type {
00455         // lock() also keeps the object alive for the duration of the call
00456         const auto obj_ = wp.lock();
00457         if (!obj_) {
00458             // inform the actuator about dead binding
00459             throw invalid_action("bind: invalid object");
00460         }
00461         return ((*obj_).*method)(std::forward<decltype(args)>(args)...);
00462     };
00463 }
00464
00485 template <typename class_t, typename T,
00486         typename action_t = std::function<typename function_remove_const<T>::type>
00487 action_t bind(class_t* obj, T class_t::* method) {
00488     return [obj, method](auto&&... args) -> typename action_t::result_type {
00489         if (!obj) {
00490             // inform the actuator about dead binding
00491             throw invalid_action("bind: invalid object");
00492         }
00493         return ((obj)->*method)(std::forward<decltype(args)>(args)...);
00494     };
00495 }
00496
00497 } // namespace untangle

```


Index

- actions__t
 - untangle::actuator< action__t >, [19](#)
- actuator, [1](#)
- add
 - untangle::actuator< action__t >, [22](#), [23](#)
- bind
 - namespace untangle: functions, [12](#)
- connect
 - namespace untangle: functions, [11](#)
- has__action
 - untangle::actuator< action__t >, [24](#)
- invalid__action
 - untangle::invalid__action, [25](#)
- invoke__action
 - untangle::actuator< action__t >, [22](#)
- invoke__callback
 - untangle::actuator< action__t >, [20](#)
- is__connected
 - untangle::actuator< action__t >, [24](#)
- last__arg
 - untangle, [15](#)
- namespace untangle: functions, [11](#)
 - bind, [12](#)
 - connect, [11](#)
- operator()
 - untangle::actuator< action__t >, [22](#)
- operator=
 - untangle::actuator< action__t >, [20](#)
- remove
 - untangle::actuator< action__t >, [23](#), [24](#)
- reset
 - untangle::actuator< action__t >, [20](#)
- results__t
 - untangle::actuator< action__t >, [19](#)
- type
 - untangle::actuator< action__t >, [20](#)
- untangle, [15](#)
 - last__arg, [15](#)
- untangle::actuator< action__t >, [17](#)
 - actions__t, [19](#)
 - add, [22](#), [23](#)
 - has__action, [24](#)
 - invoke__action, [22](#)
 - invoke__callback, [20](#)
 - is__connected, [24](#)
 - operator(), [22](#)
 - operator=, [20](#)
 - remove, [23](#), [24](#)
 - reset, [20](#)
 - results__t, [19](#)
 - type, [20](#)
- untangle::invalid__action, [25](#)
 - invalid__action, [25](#)
 - what, [26](#)
- what
 - untangle::invalid__action, [26](#)